

Маршрутизация и HTTP-методы в Express.js

Как сервер определяет, какой код выполнить для конкретного запроса

Результаты занятия

- ✓ Объяснять назначение маршрутизации в серверном приложении
- ✓ Определять маршрут по HTTP-методу и URL
- ✓ Создавать маршруты с помощью `app.get()`, `app.post()`, `app.put()`, `app.patch()` и `app.delete()`
- ✓ Выбирать HTTP-метод в зависимости от выполняемой операции
- ✓ Создавать несколько маршрутов для одного ресурса
- ✓ Проектировать базовую структуру REST API
- ✓ Проверять маршруты с помощью браузера и HTTP-клиента

Учебные вопросы

Понятие маршрутизации

Маршрут, endpoint, HTTP-метод и обработчик

HTTP-методы

GET, POST, PUT, PATCH и DELETE

Маршруты REST API

Проектирование API на примере интернет-магазина

Создание маршрутов в Express.js

Синтаксис `app.METHOD(PATH, HANDLER)` и поиск подходящего маршрута

Один ресурс — разные операции

Использование одинакового URL с разными HTTP-методами



Учебный вопрос 1. Понятие маршрутизации

Как сервер выбирает обработчик для входящего HTTP-запроса

Сервер получает разные запросы

- Одно серверное приложение обычно предоставляет множество функций.
- Клиент может запрашивать каталог, данные товара, корзину или список заказов.
- Для каждой операции сервер должен выполнить соответствующий программный код.
- Поэтому серверу необходимо определить, какому обработчику передать входящий запрос.

Главная идея

Маршрутизация связывает входящий HTTP-запрос с кодом, который должен его обработать.

Что такое маршрутизация

- Маршрутизация — механизм определения реакции серверного приложения на запрос к определённому URL с определённым HTTP-методом.
- Запрос содержит HTTP-метод, например GET или POST.
- Запрос содержит адрес ресурса, например `/api/products`.
- Express сравнивает запрос с зарегистрированными маршрутами.
- При совпадении запускается соответствующая функция-обработчик.

Основные элементы маршрута

HTTP-метод

Определяет характер операции: чтение, создание, изменение или удаление.

Path

Определяет URL-путь, для которого зарегистрирован маршрут.

Handler

Функция, которую Express запускает при совпадении метода и пути.

Response

Результат обработки, который сервер отправляет клиенту.

Анатомия маршрута Express

Маршрут = HTTP-метод + путь + обработчик



Express сопоставляет
одновременно
HTTP-метод и путь

1 app
Экземпляр
Express-приложения,
через который
регистрируются маршруты.

2 HTTP-метод
Определяет тип операции
(GET, POST, PUT, PATCH,
DELETE и др.).

3 Путь (path)
URL, для которого
регистрируется маршрут
(может содержать параметры).

4 Обработчик (handler)
Функция, которая выполняется
при совпадении HTTP-метода
и пути. Обычно принимает
параметры req и res.

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```



Внутри обработчика
можно использовать
req и res для работы
с запросом и ответом.

Как это работает

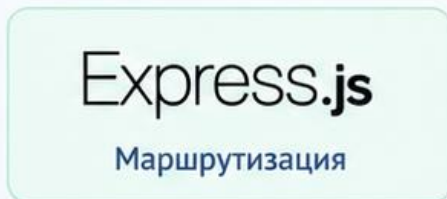
1 Клиент отправляет запрос
Например, из браузера или
HTTP-клиента



GET /api/products

HTTP-запрос

2 Express ищет маршрут
Сравнивает метод и путь
с зарегистрированными
маршрутами



Express.js

Маршрутизация

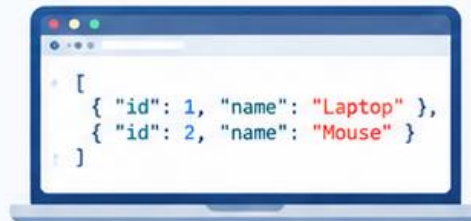
3 Выполняется обработчик
Если метод и путь совпадают,
Express запускает соответствующую
функцию



```
(req, res) => {  
  res.json(products);  
}
```

Код обработчика

4 Сервер отправляет ответ
Результат обработки возвращается
клиенту



HTTP-ответ (JSON)

```
{  
  { "id": 1, "name": "Laptop" },  
  { "id": 2, "name": "Mouse" }  
}
```



Важно: для выбора обработчика должны совпасть и HTTP-метод, и путь запроса.



Один и тот же путь может иметь разные обработчики
для разных HTTP-методов.

Примеры маршрутов будущего магазина

Метод	URL	Назначение
GET	/api/products	Получить товары
POST	/api/products	Добавить товар
GET	/api/categories	Получить категории
POST	/api/orders	Создать заказ
DELETE	/api/cart	Очистить корзину

Что такое endpoint

- Endpoint — доступная клиенту точка API, определяемая сочетанием HTTP-метода и URL.
- GET /api/products и POST /api/products — разные endpoints.
- URL у них одинаковый, но операция различается.
- Endpoint описывает конкретную возможность серверного API.
- Frontend обращается к endpoints с помощью HTTP-запросов.

Главная идея

Endpoint — это не только URL. HTTP-метод также является частью определения операции.

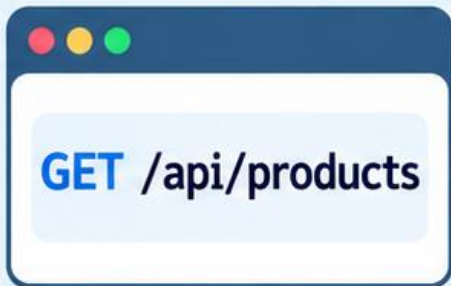
Как Express выбирает подходящий маршрут

Express последовательно проверяет зарегистрированные маршруты и ищет тот, где совпадают и HTTP-метод, и путь запроса.

Один запрос.
Нужный обработчик.
Простая логика!

1 Входящий запрос

Клиент (браузер, Postman и т.д.) отправляет HTTP-запрос на сервер Express.



Например, пользователь запрашивает список товаров

2 Express.js

Проверяет зарегистрированные маршруты по порядку. Для каждого маршрута сравниваются два условия: HTTP-метод и путь.



Маршрутизация

Запрос передается в Express.js для поиска подходящего маршрута.

3 Зарегистрированные маршруты

Express последовательно сравнивает запрос с каждым зарегистрированным маршрутом.



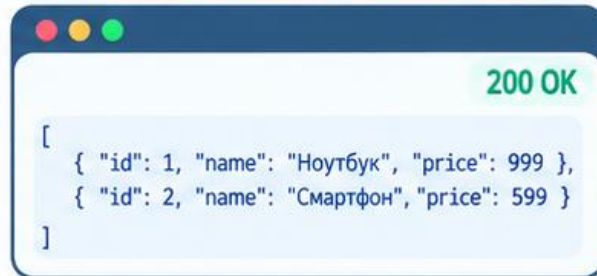
4 Обработчик маршрута

Выполняется код обработчика для совпавшего маршрута.



5 HTTP-ответ

Сервер отправляет результат клиенту.



Легенда (что проверяет Express)



Совпал метод

HTTP-метод запроса и маршрута одинаковые (GET, POST и т.д.)



Совпал путь

Путь запроса и маршрута полностью совпадают (/api/products и т.д.)



Важно: для выбора обработчика должны совпасть и HTTP-метод, и путь запроса.

Express
делает веб
проще!





Учебный вопрос 2. Создание маршрутов в Express.js

Синтаксис маршрутов и обработчики запросов

Общий синтаксис маршрута

```
app.METHOD(PATH, HANDLER);
```

Что важно

- `app` — экземпляр Express-приложения.
- `METHOD` — HTTP-метод в нижнем регистре.
- `PATH` — путь, для которого регистрируется маршрут.
- `HANDLER` — функция, которая выполняется при совпадении запроса.

Простой GET-маршрут

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

Что важно

- `app.get()` регистрирует обработчик GET-запроса.
- `/api/products` — путь к ресурсу товаров.
- `req` содержит информацию о запросе клиента.
- `res` используется для отправки ответа.
- В примере клиент получает массив `products`.

Несколько GET-маршрутов

Пример

```
app.get("/", (req, res) => {  
  res.send("Internet Shop API");  
});  
  
app.get("/api/products", (req, res) => {  
  res.json(products);  
});  
  
app.get("/api/categories", (req, res) => {  
  res.json(categories);  
});
```

Разбор

- Каждый путь представляет отдельную возможность сервера. Express выбирает обработчик в зависимости от URL входящего GET-запроса.

Маршрут определяется методом и путём

- Путь сам по себе не определяет всю операцию.
- Express учитывает HTTP-метод запроса.
- GET /api/products может возвращать каталог.
- POST /api/products может создавать новый товар.
- Поэтому один URL можно использовать для нескольких связанных операций.

Одинаковый URL — разные маршруты

GET /api/products

- Метод GET
- Получение данных
- Сервер возвращает список товаров
- Данные каталога не должны изменяться

POST /api/products

- Метод POST
- Передача новых данных
- Сервер создаёт новый товар
- Состояние приложения изменяется

Одинаковый путь с разными методами

JAVASCRIPT

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});  
  
app.post("/api/products", (req, res) => {  
  res.status(201).json({  
    message: "Product created"  
  });  
});
```

Путь маршрута и URL

- Путь начинается с символа /.
- Для API удобно использовать общий префикс /api.
- Имена ресурсов обычно записывают существительными: /products, /orders, /users.
- Связанные endpoints желательно оформлять единообразно.
- Строка запроса после символа ? не является частью пути маршрута Express.

Главная идея

Например,
`/api/products?category=phones`
соответствует маршруту
`/api/products`. Query-
параметры изучим отдельно.



Учебный вопрос 3. HTTP-методы

Как HTTP-метод передаёт смысл операции серверу

Зачем нужны разные HTTP-методы

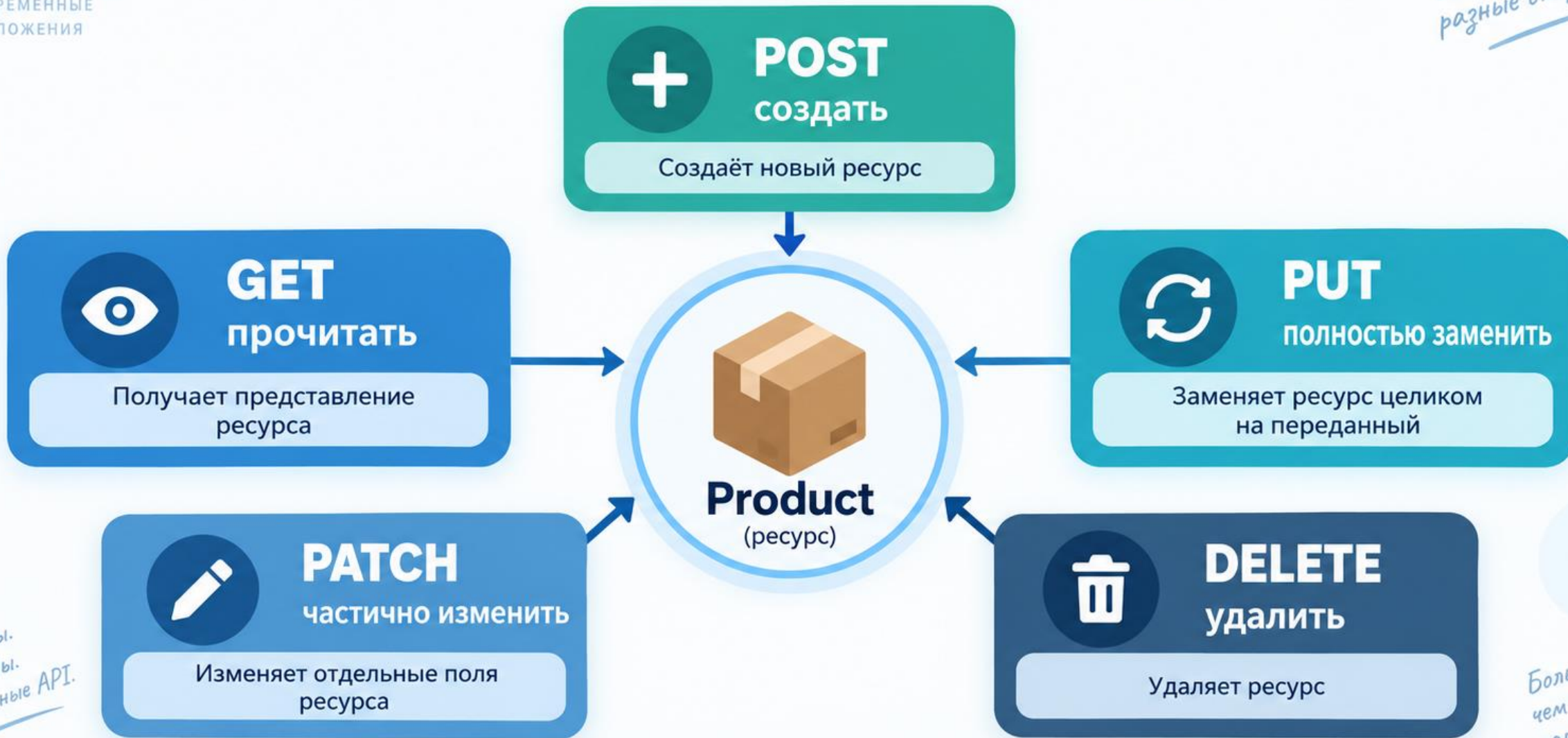
- HTTP-запрос должен сообщить серверу не только адрес ресурса, но и требуемое действие.
- Один ресурс можно получать, создавать, изменять и удалять.
- Для этих действий HTTP определяет разные методы.
- Правильный выбор метода делает API понятным и предсказуемым.

Главная идея

Метод отвечает на вопрос: что клиент хочет сделать с ресурсом?

HTTP-методы и операции с ресурсом

*Один ресурс —
разные операции*



*Ресурсы.
Методы.
Понятные API.*

*Больше,
чем просто
запросы*



HTTP-метод передаёт смысл действия над ресурсом

ПРОСТОЙ API
БОЛЬШИЕ ВОЗМОЖНОСТИ

Основные HTTP-методы REST API

Метод	Типичная операция	Пример
GET	Получить данные	Получить каталог товаров
POST	Создать ресурс или выполнить действие	Добавить новый товар
PUT	Полностью заменить ресурс	Заменить данные товара
PATCH	Частично изменить ресурс	Изменить только цену
DELETE	Удалить ресурс	Удалить товар

GET — получение данных

JAVASCRIPT

```
app.get("/api/products", (req, res) => {  
  res.json(products);  
});
```

Что важно

- GET применяется для чтения данных.
- Клиент запрашивает представление существующего ресурса.
- GET-запрос не должен использоваться для создания или удаления данных.
- Браузер легко выполняет GET-запрос при открытии URL.

POST — создание нового ресурса

JAVASCRIPT

```
app.post("/api/products", (req, res) => {  
  res.status(201).json({  
    message: "Product created"  
  });  
});
```

Что важно

- POST обычно используют для отправки новых данных серверу.
- В интернет-магазине POST может создавать товар, заказ или пользователя.
- При успешном создании ресурса часто используется статус 201 Created.
- Получение тела POST-запроса подробно рассмотрим в следующей лекции.

PUT — полная замена ресурса

JAVASCRIPT

```
app.put("/api/products", (req, res) => {  
  res.json({  
    message: "Product replaced"  
  });  
});
```

Что важно

- PUT предназначен для замены представления ресурса новыми данными.
- Клиент обычно передаёт полный набор изменяемых данных ресурса.
- В реальном API изменяемый объект обычно определяется идентификатором в URL.
- Параметры пути будут рассмотрены на следующем занятии.

PATCH — частичное изменение

JAVASCRIPT

```
app.patch("/api/products", (req, res) => {  
  res.json({  
    message: "Product partially updated"  
  });  
});
```

Что важно

- PATCH используется, когда требуется изменить только часть данных.
- Например, можно изменить цену товара, не передавая его название и категорию заново.
- PATCH особенно удобен для частичного редактирования сущностей.
- Сами изменяемые поля клиент передаёт в теле запроса.

PUT и PATCH

PUT

- Полная замена представления ресурса
- Обычно передаётся полный набор данных
- Используется, когда новое состояние известно целиком
- Пример: полностью заменить карточку товара

PATCH

- Частичное изменение ресурса
- Передаются только изменяемые данные
- Остальные значения сохраняются
- Пример: изменить только цену товара

DELETE — удаление ресурса

JAVASCRIPT

```
app.delete("/api/products", (req, res) => {  
  res.json({  
    message: "Product deleted"  
  });  
});
```

Что важно

- DELETE применяется для удаления ресурса.
- В реальном API обычно необходимо определить, какой именно объект нужно удалить.
- Для этого часто используется идентификатор в URL.
- Перед удалением сервер также может проверять права пользователя.

HTTP-метод должен соответствовать смыслу операции

Чтение

Используйте GET, если клиент только получает данные.

Создание

Используйте POST для создания нового ресурса.

Обновление

Используйте PUT или PATCH в зависимости от характера изменения.

Удаление

Используйте DELETE, если ресурс необходимо удалить.

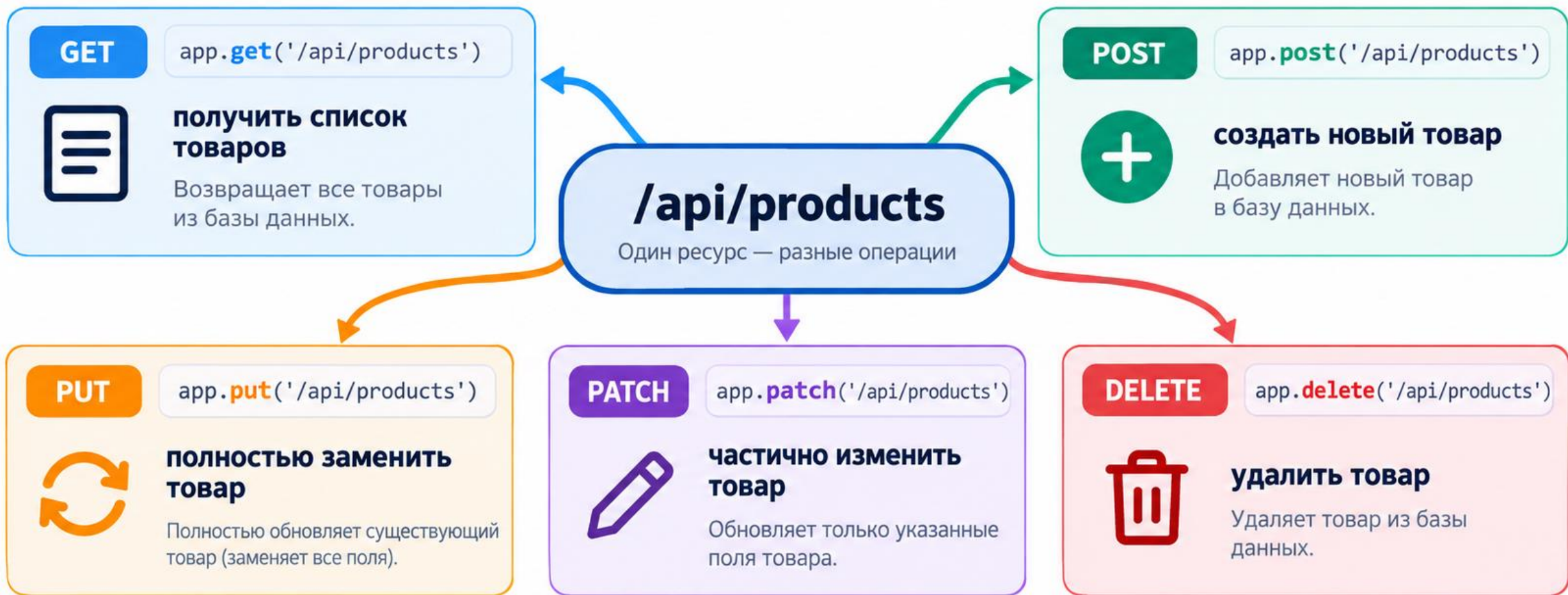


Учебный вопрос 4. Один ресурс — разные операции

Связываем действия над сущностью с HTTP-методами

Один ресурс **products** — пять HTTP-операций

Один и тот же URL может обрабатывать разные запросы в зависимости от HTTP-метода.



ключевой вывод

Одинаковый путь не означает одинаковый маршрут:
HTTP-метод определяет операцию.

Маршруты ресурса products

HTTP-запрос	Смысл	Express
GET /api/products	Получить товары	app.get()
POST /api/products	Создать товар	app.post()
PUT /api/products	Полностью заменить данные	app.put()
PATCH /api/products	Частично изменить данные	app.patch()
DELETE /api/products	Удалить ресурс	app.delete()

Набор маршрутов для products

JAVASCRIPT

```
app.get("/api/products", getProducts);  
app.post("/api/products", createProduct);  
app.put("/api/products", replaceProduct);  
app.patch("/api/products", updateProduct);  
app.delete("/api/products", deleteProduct);
```

Обработчик можно вынести в функцию

JAVASCRIPT

```
function getProducts(req, res) {  
  res.json(products);  
}  
  
app.get("/api/products", getProducts);
```

Что важно

- Обработчик не обязательно писать непосредственно внутри `app.get()`.
- Именованная функция делает код маршрутов короче.
- Логику проще повторно использовать и тестировать.
- Позже обработчики будут вынесены в отдельные контроллеры.

Маршрут и бизнес-логика — разные задачи

- Маршрут определяет, какой запрос должен быть обработан.
- Обработчик выполняет действия, необходимые для ответа.
- Бизнес-логика определяет правила приложения.
- По мере роста проекта эти части желательно разделять.
- На первых примерах допустимо хранить небольшую логику прямо в обработчике.

Главная идея

Маршрут отвечает на вопрос «куда направить запрос», а не содержит всю архитектуру приложения.



Учебный вопрос 5. Маршруты REST API

Проектируем понятные endpoints для учебного интернет-магазина

Ресурсный подход

- REST API удобно проектировать вокруг ресурсов предметной области.
- В интернет-магазине ресурсами могут быть products, categories, users, carts и orders.
- URL называет ресурс, а HTTP-метод описывает действие.
- Такой подход уменьшает количество специальных URL и делает API предсказуемым.

Главная идея

**Лучше POST /api/products, чем
GET /api/createProduct.**

Действие в URL или HTTP-метод

Менее удачно

- GET /api/getProducts
- POST /api/createProduct
- POST /api/deleteProduct
- Действие дублируется в названии URL

Ресурсный подход

- GET /api/products
- POST /api/products
- DELETE /api/products
- Смысл операции задаётся HTTP-методом

Черновая карта API интернет-магазина

Ресурс	GET	POST	Другие операции
/api/products	Получить товары	Создать товар	PUT, PATCH, DELETE
/api/categories	Получить категории	Создать категорию	Изменение и удаление
/api/orders	Получить заказы	Создать заказ	Изменение статуса
/api/cart	Получить корзину	Добавить данные	Изменение и очистка

HTTP-статус дополняет ответ

- HTTP-метод описывает требуемую операцию.
- HTTP-статус сообщает результат выполнения операции.
- GET при успешном чтении обычно возвращает 200 OK.
- POST при успешном создании ресурса часто возвращает 201 Created.
- Ошибки также должны передаваться корректными HTTP-статусами.

Примеры методов и статусов

Ситуация	Метод	Возможный статус
Каталог успешно получен	GET	200 OK
Товар создан	POST	201 Created
Товар изменён	PUT / PATCH	200 OK
Товар удалён	DELETE	200 OK или 204 No Content
Ресурс не найден	Любой подходящий метод	404 Not Found

Как проверять разные HTTP-методы

- Адресная строка браузера удобна прежде всего для GET-запросов.
- Для POST, PUT, PATCH и DELETE нужен HTTP-клиент или программный frontend.
- В учебных работах можно использовать Postman, Bruno или Thunder Client.
- HTTP-клиент позволяет выбрать метод, URL, заголовки и тело запроса.
- На этом занятии достаточно проверить соответствие метода и маршрута.

Проверка маршрута через HTTP-клиент

Один URL → разные HTTP-методы → разные обработчики



HTTP-клиент позволяет отправлять запросы с любыми методами и видеть ответ сервера.

HTTP-клиент

Postman / Thunder Client / Bruno и др.

GET

GET

POST

PUT

PATCH

DELETE

http://localhost:3000/api/products

Send

HTTP-запрос

GET /api/products



Express.js

Сервер
(порт 3000)

HTTP-ответ

200 OK
(JSON)

Response

Pretty

Raw

Headers

200 OK

123 ms

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 120000
  },
  {
    "id": 2,
    "name": "Mouse",
    "price": 8000
  }
]
```

Тот же URL → разные методы → разные результаты

GET → получить список

Получение данных о товарах

GET

http://localhost:3000/api/products

200 OK

```
[
  { "id": 1, "name": "Laptop" },
  { "id": 2, "name": "Mouse" }
]
```

POST → создать

Создание нового товара

POST

http://localhost:3000/api/products

201 Created

```
{
  "message": "Product created",
  "id": 3
}
```

PUT → полностью заменить

Полная замена данных товара

PUT

http://localhost:3000/api/products

200 OK

```
{
  "message": "Product replaced"
}
```

PATCH → частично изменить

Частичное обновление товара

PATCH

http://localhost:3000/api/products

200 OK

```
{
  "message": "Product updated"
}
```

DELETE → удалить

Удаление товара

DELETE

http://localhost:3000/api/products

200 OK

```
{
  "message": "Product deleted"
}
```



Важно!

Один и тот же URL может обрабатывать разные операции.
Express выбирает обработчик на основе сочетания HTTP-метода и пути.



Используйте HTTP-клиент

Браузер удобен только для GET-запросов.
Для POST, PUT, PATCH и DELETE используйте Postman, Thunder Client, Bruno или аналогичные инструменты.



Проверяйте ответ

- ✓ HTTP-статус (200, 201 и др.)
- ✓ Содержимое ответа (JSON)
- ✓ Соответствие ожидаемой операции

Мини-практика

Создайте пять маршрутов для одного ресурса `/api/products` и проверьте каждый маршрут HTTP-клиентом.

Задание

- GET `/api/products` — вернуть массив товаров
- POST `/api/products` — вернуть сообщение о создании товара
- PUT `/api/products` — вернуть сообщение о полной замене данных
- PATCH `/api/products` — вернуть сообщение о частичном изменении
- DELETE `/api/products` — вернуть сообщение об удалении

Критерии проверки

- Все пять маршрутов зарегистрированы
- Каждый HTTP-метод вызывает свой обработчик
- GET возвращает данные
- POST возвращает статус 201
- Ответы имеют JSON-формат

Контрольные вопросы

1. Что называется маршрутизацией в серверном приложении?
2. Из каких основных частей состоит маршрут Express?
3. Почему GET /api/products и POST /api/products являются разными endpoints?
4. Как выглядит общий синтаксис маршрута Express?
5. Для каких операций обычно используется метод GET?
6. В каком случае применяется POST?
7. Чем PUT отличается от PATCH?
8. Для чего используется DELETE?
9. Почему действие лучше задавать HTTP-методом, а ресурс — URL?
10. Почему для проверки POST, PUT, PATCH и DELETE удобно использовать HTTP-клиент?

Итоги занятия

1

Маршрутизация связывает HTTP-запрос с функцией-обработчиком.

2

Маршрут Express определяется сочетанием HTTP-метода и пути.

3

Основные методы REST API — GET, POST, PUT, PATCH и DELETE.

4

GET используется для чтения, POST — для создания, PUT и PATCH — для изменения, DELETE — для удаления.

5

Один URL может иметь несколько маршрутов с разными HTTP-методами.

6

REST API удобно проектировать вокруг ресурсов предметной области.

7

HTTP-клиент позволяет независимо проверять маршруты backend-приложения.